Clinical Scope – User Guide

Clinical Scope Team

November 9, 2026

Contents

1 Introduction **3**
1.1 Key Features . . . 3

2 Launching the Application **4**
2.1 First launch . . . 4
2.2 Starting the App . . . 4
2.3 Trying the Demo Dataset . . . 5
2.4 Application Overview . . . 6

3 Patient Data & Supported Data Sources **7**
3.1 Example Folder Layouts . . . 7
3.2 Folder Naming Rules . . . 7
3.3 Canonical Data Source Table . . . 7
3.4 Generic "Other" Data Source . . . 9

4 Loading Database Options **11**
4.1 Option 1: Upload a Custom Configuration File . . . 11
4.2 Option 2: Reload Last Config (Daily Workflow) . . . 11
4.3 Option 3: Default Visualization (Quick Start) . . . 11

5 Configuring Patient Options **12**
5.1 Global Options . . . 12
5.2 Per-Source Options . . . 12

6 Settings **14**
6.1 App behavior . . . 14
6.2 Plot defaults . . . 14

7 Processing Data **16**
7.1 Process Visualization (orange button) . . . 16
7.2 Inspect Data (teal button) . . . 16
7.3 Inspecting only the signals you configured . . . 17
7.4 Inspecting from the Command Line . . . 18

8 Interacting with Plots **19**
8.1 Navigation Controls . . . 19
8.2 Dynamic Resampling (FigureResampler) . . . 19

9 Annotations **20**
9.1 Annotation Toolbar . . . 20
9.2 Annotation Types . . . 20

9.3 Moving an Annotation . . . 20
9.4 Dragging an Annotation with the Mouse . . . 20
9.5 Hiding an Annotation . . . 21
9.6 Groups . . . 21
9.7 Persistence . . . 21
9.8 Python API . . . 21

10 Configuration File Reference **23**
10.1 patient_options.json . . . 23
10.2 database_options.json . . . 24
10.3 database_options.xlsx . . . 35
10.4 Derived signals . . . 41

11 Troubleshooting **45**
11.1 Browser Does Not Open Automatically . . . 45
11.2 No Data Found . . . 45
11.3 Slow Loading . . . 45
11.4 Time Alignment Issues . . . 45
11.5 Application Crashes or Errors . . . 45
11.6 Known limitations . . . 45

1 Introduction

Clinical Scope is an interactive dashboard for exploring, comparing, and annotating clinical physiological signals from multiple medical devices in a single interface.

1.1 Key Features

- **Multi-source visualization**: Display signals from FluxMed, Mindray, EIT, Servo-U and EDF recorders simultaneously.
- **Generic "Other" data source**: Drop any CSV or Parquet file with a datetime column into an `other/` folder — monitor exports, syringe pumps, anything tabular. Signals are auto-discovered and configured per file.
- **Interactive plots**: Zoom, pan, and explore data at any time scale with automatic resampling.
- **Data inspection**: Preview available columns, point counts, and time ranges for every source before running the full visualization — exportable to CSV.
- **Annotations**: Draw lines and rectangles directly on plots to mark events or regions of interest. Annotations are saved and persist across sessions.
- **Flexible configuration**: Choose which signals to display, customize labels, units, colors, and group related signals together — in JSON or Excel format.
- **Cross-datasource phase loops**: Define loop entries to build phase plots that combine signals from different devices.
- **Spectrograms**: Render a time-vs-frequency-vs-power view of any signal (e.g. EEG) over a configurable frequency band, with a fixed colour range for consistent reading across patients.
- **Power spectral densities (PSD)**: Render power against frequency, averaged over the loaded time range, with several signals overlaid on one plot for comparison.
- **Per-datasource timezone control**: Override the timezone of any supported source via `additional_informations.timezone` (all plots still render in the configured display timezone).
- **Export**: Generate standalone HTML visualizations for sharing.

The full list of supported data sources — with folder keywords, accepted file extensions, and typical signals — lives in Section 3: **Patient Data & Supported Data Sources**.

2 Launching the Application

2.1 First launch

ClinicalScope is not code-signed, so every operating system interrupts the **first** launch with a warning about an unknown publisher. This is expected. The steps below get you past it, and they are needed only once — afterwards the application starts normally.

2.1.1 Windows

Unzip the archive, then **right-click `ClinicalScope.exe` → Run as administrator**. The first launch needs elevation; a plain double-click will not start it.

SmartScreen may also warn that the publisher is unknown — choose **More info → Run anyway**.

This is a one-time step. After that first run, ClinicalScope starts on a normal double-click, including after a reboot.

2.1.2 macOS (Apple Silicon)

macOS quarantines downloaded applications and will not open an unsigned one through the normal flow. Remove the quarantine flag from the `.zip` **before** unzipping it:

```
cd ~/Downloads
xattr -d com.apple.quarantine ClinicalScope-macOS-arm64.zip
unzip ClinicalScope-macOS-arm64.zip -d ClinicalScope
```

Then run the `ClinicalScope` executable inside the `ClinicalScope` folder.

> **Heads-up:** some browsers (Safari in particular) unzip downloads automatically. If yours does, turn that option off or use another browser — the command above must run on the `.zip` file, not on an already-extracted folder. If you have downloaded the archive more than once, check that you are acting on the right `.zip` and the right extraction folder.

There is no Intel Mac build; on those machines, install with `pip install clinical-scope` instead.

2.1.3 Linux

Unzip the archive and run the executable, making it executable first if it does not start:

```
unzip ClinicalScope-linux-x86_64.zip
chmod +x ClinicalScope/ClinicalScope        # only if needed
./ClinicalScope/ClinicalScope
```

2.2 Starting the App

Locate the **ClinicalScope** executable in the application folder and double-click it. The very first time, follow First launch instead — on Windows the executable has to be started as administrator once.

A terminal window will appear showing the application starting up. After a few seconds, your default web browser will automatically open at:

`http://127.0.0.1:8050`

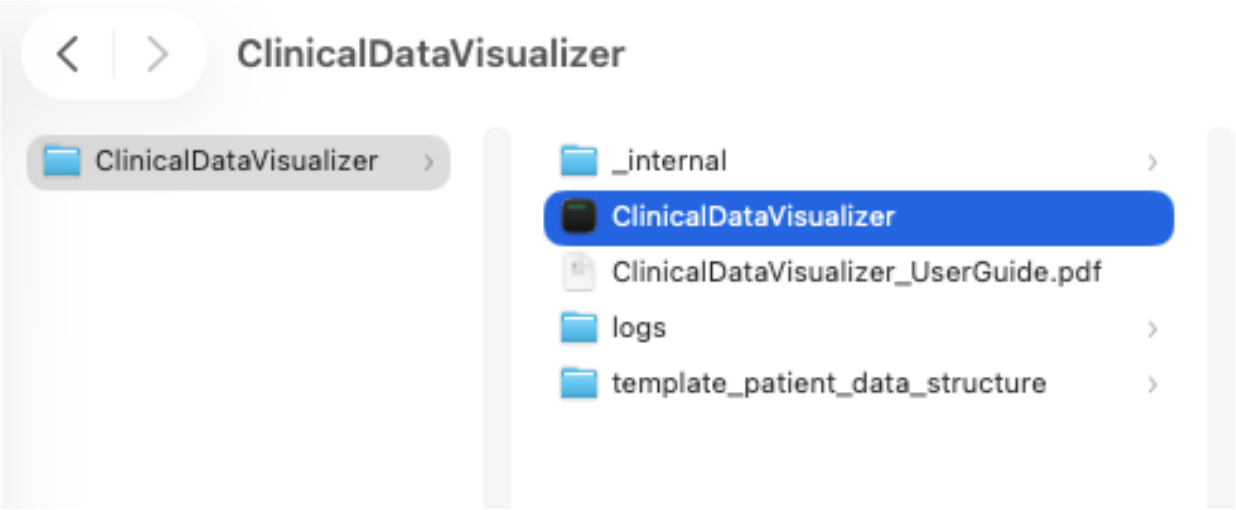


Figure 1: Application launch screen

The bundle also ships this user guide and a template folder for organizing patient data. A `logs` folder appears after the first run, to help with debugging.

To **close** ClinicalScope, close the terminal window that opened with it — the application runs inside that window. If the window is hidden, end the `ClinicalScope` process from your system's process manager.

2.3 Trying the Demo Dataset

ClinicalScope comes with a small demo recording — a single patient, with one example of every supported data source — so you can see a complete visualization before preparing any data of your own.

With the standalone application, the demo is already there: look for the `demo_database/` folder next to the executable.

With a `pip` install, the demo is not part of the package. Ask for its download link from a terminal:

```
clinical-scope --demo
```

Open the printed link in your browser. The archive — about 2 MB — lands in your Downloads folder; unzip it there and you get a single `clinical-scope-example` folder. The command also prints the two paths the app asks for, ready to complete with wherever you unzipped it. Run `clinical-scope --help` to see everything the command line offers.

Then, in the application:

1. Click **Default visualization (all sources)**, or upload the demo's `database_options.json`.
2. Set **Data folder** to the demo's `demo_patient` folder.
3. Set the EIT **day** to `2004-09-15`, and the EDF **recording start** to `2004-09-15 10:12:33`. Neither file carries the recording date and both expect you to supply one: EIT device files record time of day only, and the demo's EDF was de-identified, which blanks the date in its header.
4. Click **Inspect data** to check that every source was found and covers the period you asked for. This is worth doing on any new folder before plotting anything.
5. Click **Process visualization**.

The demo is synthetic data for learning the interface. It is not a real recording, and nothing in it should be read clinically.

2.4 Application Overview

The interface is organized top-to-bottom in the following order:

1. **Database Options** – Three buttons side-by-side:
   - **Upload config file** (blue) – load a custom `database options` file, either `.json` or `.xlsx`.
   - **Reload last config** (grey) – appears only if a previously uploaded config was cached; restores it with one click.
   - **Default visualization (all sources)** (green) – enables every registered data source with default settings, no file needed.
2. **Patient Options** – Configure data folder, time range, and per-source settings. The form is generated dynamically from the loaded database options.
3. **Action buttons** – **Process visualization** (orange) and **Inspect data** (teal).
4. **Annotations Controls** – Shape dropdown + Modify/Delete buttons (visible after a successful visualization).
5. **Inspection pop-up** – Full-screen overlay triggered by the Inspect button, with per-source status badges, column tables, and a CSV download.
6. **Visualization Area** – Interactive plots.

A **Settings** button sits at the top right, above the Database Options row. It opens your personal display and export settings, which apply to every patient you open — see Settings. Below it, **Docs** opens this guide in a new browser tab, at the version you are running — in the standalone application, the copy that shipped with it, so it opens with no connection needed.

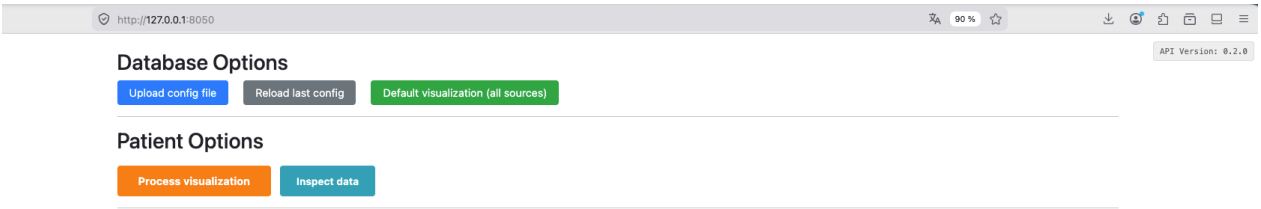


Figure 2: Application main interface

3 Patient Data & Supported Data Sources

Each patient's data must be organized in a root folder with **one subfolder per data source**. The application automatically identifies data sources based on keywords in subfolder names.

You only need subfolders for the data sources you actually have — empty or missing subfolders are silently skipped. The `clinical_scope_output/` subfolder is created automatically the first time you process a patient. It contains annotations, `.html` visualizations, formatted data.

3.1 Example Folder Layouts

A full setup with several devices:

```
Patient1/
  eit/
  fluxmed_signals/
  fluxmed_parameters/
  servo_u/
  mindray_scope/
  edf/
  other/
  clinical_scope_output/                ← auto-created
```

An **"Other"-heavy** setup — drop any CSV/Parquet file with a datetime column into `other/`; each file is configured independently via `other::<stem>` keys in the database options (see dedicated section below):

```
Patient1/
  other/
    waves.parquet          → configured under "other::waves"
    numerics.csv           → configured under "other::numerics"
    syringe_log.csv        → configured under "other::syringe_log"
  clinical_scope_output/
```

3.2 Folder Naming Rules

Folder names are **flexible** — they just need to contain the required keywords:

- **Case-insensitive** — `Mindray_Scope`, `MINDRAY-SCOPE`, `mindray scope` all match.
- **Any separator** — underscore, dash, space, or none.
- **Any order** — `scope_mindray` works just as well as `mindray_scope`.
- **Partial keywords don't match** — `flux` alone will not match `fluxmed_*`; the full word is required.

A few sources are also **identified by file extension** when the folder is ambiguous or generic (e.g., `.asc` files inside a folder are enough to classify it as EIT, `.sta` for Servo-U, `.xml` for Mindray Scope).

3.3 Canonical Data Source Table

Source	Module name	Folder keywords	Accepted extensions (ordered by preference)	Discovery mode	Typical signals
EIT (PulmoVista)	`eit`	`eit`	`.asc`	All files	Global/local impedance, impedance percentages
FluxMed Signals	`fluxmed_signals`	`fluxmed`, `signals`	`.parquet`, `.txt`, `.csv`	Single file	Respiratory waveforms

Source	Module name	Folder keywords	Accepted extensions (ordered by preference)	Discovery mode	Typical signals
FluxMed Parameters	`fluxmed_parameters`	`fluxmed`, `parameters`	`.parquet`, `.txt`, `.csv`	Single file	Respiratory parameters
Servo-U	`servo_u`	`servo`	`.sta`	All files	Ventilator waveforms and settings
Mindray Scope	`mindray_scope`	`mindray`	`.xml`, `.csv`	All files	Monitor waveforms (ECG, SpO2, pressure)
Mindray Respi Waves	`mindray_respi_waves`	`mindray`, `resp`, `wave`	`.parquet`, `.csv`	Single file	High-frequency respiratory waveforms
Mindray Respi Numerics	`mindray_respi_numerics`	`mindray`, `resp`, `numeric`	`.parquet`, `.csv`	Single file	Respiratory parameters (Vt, RR, PEEP, etc.)
EDF / EDF+	`edf`	`edf`	`.edf`	All files	Any signal an amplifier or recorder exports as EDF (typically EEG)
Other (Generic)	`other`	`other`	`.parquet`, `.csv`	All files (one entry **per file**)	Any time-series with a datetime column

Single file sources expect exactly one data file per folder. When several formats coexist (e.g., `data.csv` and `data.parquet`), the most preferred extension wins. If multiple unrelated stems remain after that filter, the source is skipped and a warning is logged.

All files sources load every matching file in the folder and concatenate them. The `Other` source is special: each file produces an independent entry named `other::<stem>` (see below).

EDF recordings. Channels are read exactly as the file stores them — a file holding bipolar derivations (`Fp1-F7`) shows those, a file holding referential channels shows those. Channels recorded at different sample rates are placed on a shared time axis, each keeping its own sampling.

An EDF header always states a start date and a start time, but de-identification commonly blanks them (the format's "unknown date" is `01.01.1985`), leaving a recording that is effectively relative time only. Place such a recording with the per-source `recording_start` option in patient_options.json:

- **A full timestamp** (`2024-10-08 08:12:33`) puts the first sample at that instant — use this when the file's clock time was blanked too.
- **A date alone** (`2024-10-08`) shifts the recording by whole days and keeps the file's own time of day — use this when only the date was scrubbed.
- Times are read in the **device's** timezone (the source's `additional_informations.timezone`, `Europe/Paris` by default), not your display timezone.

A file that still carries a real start date keeps it, and `recording_start` is ignored. A file with no date and no `recording_start` is still plotted, anchored at 1985-01-01, with a warning in the log.

Per-source configuration options (`field_display`, `signals`, `grouped_fields`, `loop`, `additional_informations`, etc.) are documented in the Configuration File Reference section.

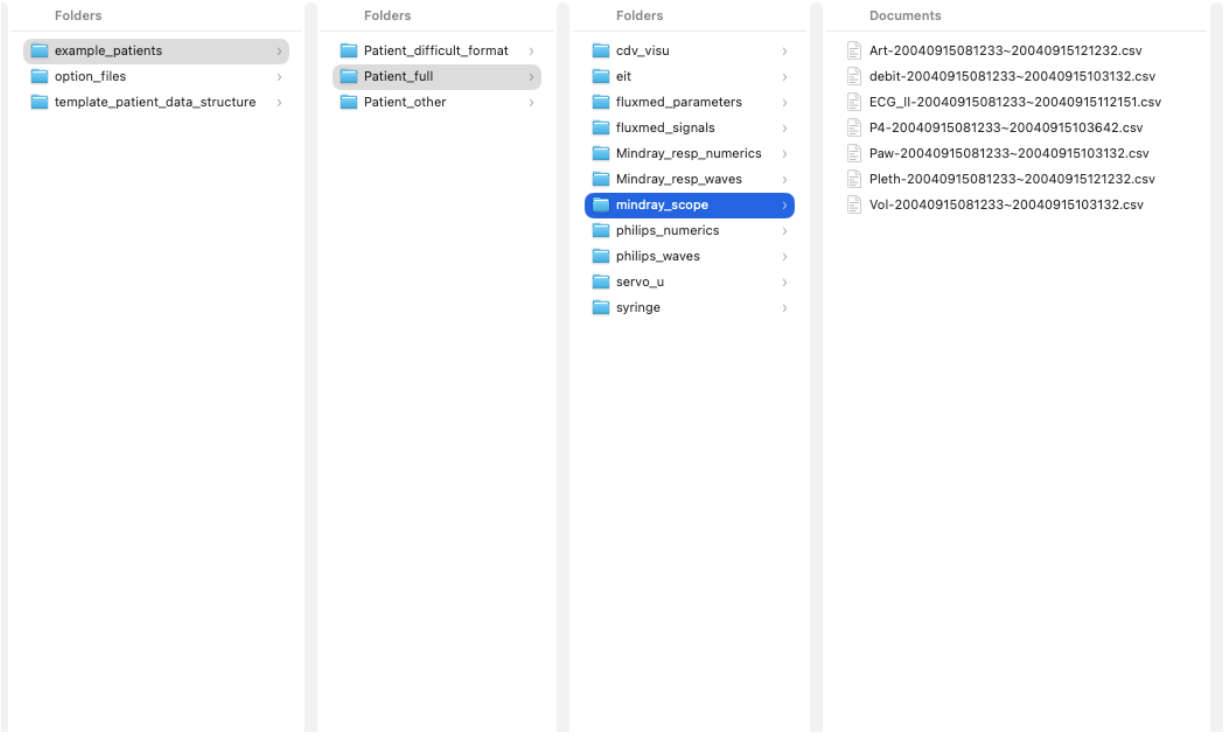


Figure 3: Patient folder structure example

3.4 Generic "Other" Data Source

`other` is the **generic escape hatch** for any CSV or Parquet file that has a datetime column but does not fit any of the specialised sources — the most natural entry point if your data is already well formatted.

How it works:

- Drop any number of `.csv` or `.parquet` files into an `other/` subfolder; every one is discovered automatically.
- Each file produces an **independent entry** keyed by its stem (filename without extension): `waves.parquet` becomes `other::waves`.
- Columns within a file are exposed as `<stem>::<column_name>` so names stay globally unique (important for cross-datasource groups and loops).

Datetime column auto-detection. The loader automatically finds your file's datetime column by name and content, including common non-English names — no configuration needed. If no column can be confidently identified as a timestamp, the file is skipped (with a warning) rather than guessed at.

Per-file configuration. In `database_options`, use `other::<stem>` keys — one block per file — exactly like any other datasource:

```
"other::waves": {
    "field_display": ["art", "pleth"],
    "signals": {
        "art":   { "label": "Arterial Pressure", "unit": "mmHg", "color": "red" },
        "pleth": { "label": "Plethysmography",  "unit": "AU",   "color": "purple" }
    },
    "additional_informations": { "timezone": "Europe/Paris" }
},
"other::numerics": {
```

```
    "field_display": ["FC", "SpO2"]
}
```

Per-file plots. A per-file block may also define `grouped_fields`, `loop`, `spectrogram` and `psd`, naming its own columns directly — no `other::<stem>::` prefix needed inside the block:

```
"other::eeg": {
    "spectrogram": {
        "Fp1": { "signal": "Fp1", "freq_range": [0.5, 30.0] }
    }
}
```

The plot is titled with its file's name in front — the example above appears as `eeg::Fp1` — so two files can each define a `PV` loop or an `Fp1` spectrogram without one replacing the other.

Per-file timezone. Each `other::<stem>` block may declare its own `additional_informations.timezone`, for a file exported by a device in another zone. Without it, timestamps that carry no timezone of their own are read as UTC.

Per-file trace style. A `trace_options` block changes how that file's traces are drawn — sparse, step-like data (infusion rates, hand-entered values) reads much better with visible points than as a bare line:

```
"other::syringe": {
    "trace_options": { "mode": "lines+markers", "line_width": 2.0 },
    "additional_informations": { "timezone": "Europe/Paris" }
}
```

The same block works in any per-source section; each `other::<stem>` file carries its own, so a curated infusion log and a raw waveform export can look different. Full key list in `trace_options` Block.

Per-file processing options. Every file you declare with an `other::<stem>` key also gets its own box in the *Specific Options* panel, sitting alongside the device boxes rather than nested under a shared "Other" one. Each box carries that file's own `time_shift` and *Group signals by source file*, so a curated two-column export and a ninety-column raw dump can be corrected and laid out independently. Files present in the folder but not declared in `database_options` fall back to the shared "Other (generic)" box. See patient_options.json.

Inspection shows one entry per file, so you can verify that every file was correctly discovered and parsed — see Inspect Data.

See `example/demo_database/database_options.json` in the source repository for a full reference configuration — its `other::waves`, `other::numerics` and `other::syringe` sections configure the three files shipped in `demo_patient/other/`.

4 Loading Database Options

Database options define **which data sources to enable** and **how signals should be displayed** (labels, units, colors, grouping). There are three ways to get one into the app.

4.1 Option 1: Upload a Custom Configuration File

Click the blue **"Upload config file"** button to load a custom configuration. Two formats are accepted:

- **`.json` extension** — a JSON file following the structure described in Configuration File Reference section.
- **`.xlsx` extension** — an Excel spreadsheet following the column layout documented in the same reference. The spreadsheet is converted to the equivalent JSON structure on load.

When the upload succeeds the file is **cached locally** at `~/.clinical_scope/last_database_options.json` — this is what powers Option 2 below. A database options file holds only signal metadata (labels, colors, units, field mappings), never patient data or PHI, so nothing sensitive leaves the patient folder.

4.2 Option 2: Reload Last Config (Daily Workflow)

If a custom configuration was previously uploaded, a grey **"Reload last config"** button appears automatically on startup. Click it to instantly restore the last used configuration without browsing for files — ideal when you re-open the app every day with the same setup.

4.3 Option 3: Default Visualization (Quick Start)

Click the green **"Default visualization (all sources)"** button. This automatically enables **every registered data source** with its built-in default display settings — no configuration file needed. This is the recommended starting point for new users, and it automatically picks up any new data sources added to the library without requiring a config update. Note that if you have hundreds of timeseries, they will all be plot and you may experience performance issues on the app.

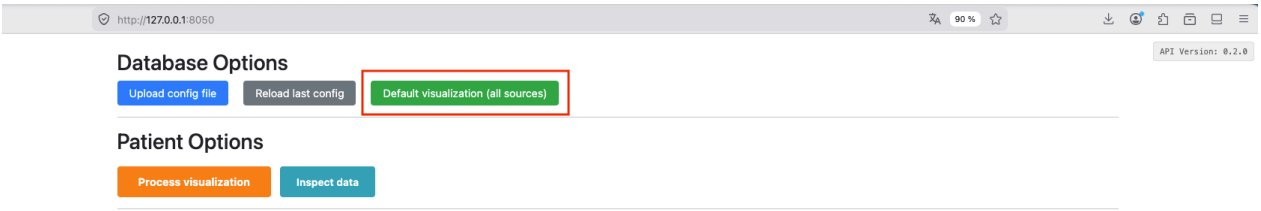


Figure 4: Default visualization button

5 Configuring Patient Options

After loading database options, the **Patient Options** form appears, generated dynamically from them.

5.1 Global Options

These apply to all data sources:

Option	Description
Path to data (folder)	Full path to the patient's root data folder
Time start filter	Start of the time window to display (format: `YYYY-MM-DD HH:MM:SS`). Leave empty to use all available data.
Time end filter	End of the time window to display. Leave empty to use all available data.
Re-use data if already loaded once	When checked, reuses previously cached `.parquet` files from the `clinical_scope_output/` folder, significantly speeding up subsequent loads. **Un-tick** if raw patient data has been modified, or once after updating ClinicalScope

The time filters are typed and shown in the **Display timezone**, set once in Settings — a label next to the fields names it.

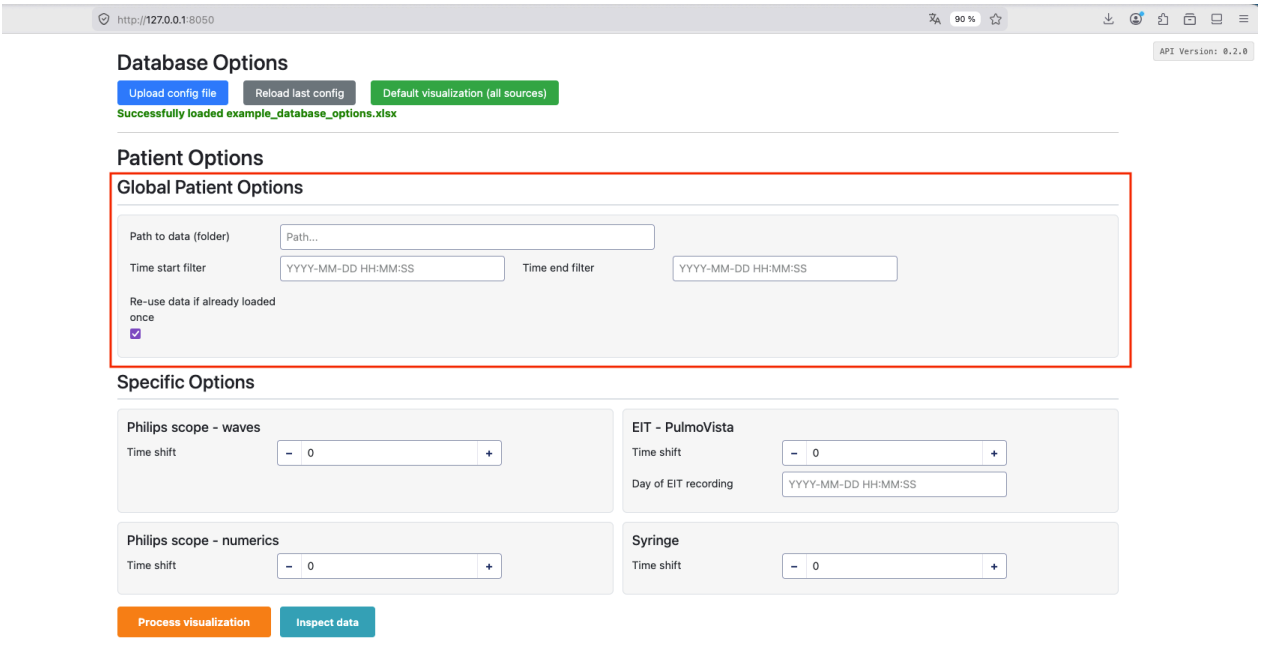


Figure 5: Global patient options

5.2 Per-Source Options

Each data source present in the loaded database options gets its own card below the global options. Common per-source options:

- **Time shift** (seconds): Adjust the time alignment of a source relative to others. Useful when devices were not perfectly synchronized.
- **Day**: Specify the recording date for sources that require it (e.g., EIT data).

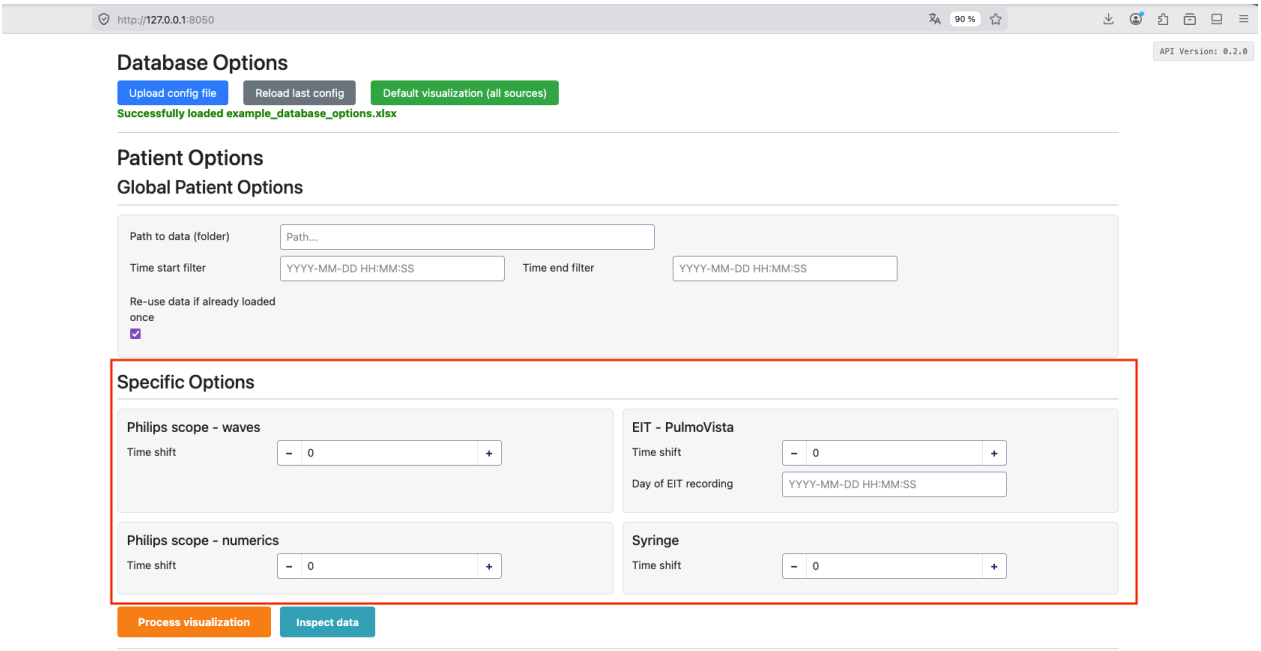


Figure 6: Per-source options

6 Settings

The **Settings** button at the top right opens your personal settings. They belong to you rather than to a patient or a database: they stay the same whichever data you open, and are saved as soon as you change them, so they survive a restart.

Your settings never overwrite a database options file. They fill the gaps: if the configuration sets a color for a signal, that color is used; if it says nothing, your palette applies. Changes take effect on the next **Process visualization**.

6.1 App behavior

Setting	What it does
Save a full-resolution HTML export on each Process	Writes `visualization.html` into `clinical_scope_output/` every time you process. Off by default, because the export takes extra time on large recordings.
Embed Plotly in the HTML export	Makes the exported file self-contained, so it opens on a machine with no internet access. The file grows by roughly 3.5 MB. Leave it off if the file is only ever opened online.
Inspect: read only configured signals	Makes **Inspect data** read only the signals your database options list, instead of everything in the file — lighter on memory, at the price of no longer seeing the signals you have not configured yet. Off by default (see Inspecting only the signals you configured).

6.2 Plot defaults

Setting	What it does
Display timezone	The timezone every plot and the Patient Options time filters are shown in (IANA name, e.g. `Europe/Paris`). Changing it does not move your data — the time filters are rewritten to keep pointing at the same instant, just written in the new timezone's local time.
Height of each time-series subplot	In pixels.
Height of each loop subplot	Same, for loop plots — which stay square, so this also sets their width.
Loop subplots per row	How many loops sit side by side, 1 to 3.
Maximum width of one legend entry	Caps how much horizontal room the legend can take from the plot. Longer signal names are truncated.
Palette for signals with no color in the config	Both colorblind-safe palettes are readable under the common color-vision deficiencies.
Plot theme	Light or dark background.
Hover: x-axis time format	Whether the hover panel shows the time only, or the full date and time.

Setting	What it does
Hover: panel style	*Unified* lists every trace at the hovered position in one panel; *closest point only* shows just the nearest trace. It applies to plots that can have multiple traces. Unified is comfortable for a few signals and crowded on a subplot with many.
Hover: significant digits of the y value	How precisely hovered values are printed. A signal with its own hover format in the database options keeps it.
Spectrogram colour range — minimum / maximum (dB)	Colour scale bounds used by any spectrogram whose configuration leaves `db_range` unset.

7 Processing Data

Once patient options are configured, two actions are available from the action row. **Both run the same data pipeline** — find → load → format — and share the same per-datasource progress bar. What differs is only the outcome: the orange button builds interactive plots, the teal button produces a structured summary of what was found.

7.1 Process Visualization (orange button)

Click the large orange **"Process visualization"** button to generate the plots. The steps are:

1. **Validation**: The application verifies that all mandatory fields are filled in and that the data folder exists.
2. **Data Discovery**: For each enabled data source, the application scans the patient folder for matching subfolders and files.
3. **Data Loading**: Raw data files are parsed according to each source's format.
4. **Formatting**: Signals are filtered, resampled, and converted using your database options (labels, units, time range).
5. **Caching**: Processed data is saved as `.parquet` files in the `clinical_scope_output/` subfolder for faster reloading next time. Tick **"Re-use data if already loaded once"** (`quick_load`) to skip raw parsing on subsequent runs and read the cache instead.
6. **Plot Generation**: Interactive Plotly figures are created and displayed in the visualization area.

While processing runs, a **per-datasource progress bar** appears below the action row. It shows `(completed / total): <datasource>` and its color matches the action — orange for visualization. The bar advances as each datasource completes; it never reaches 100% while the last source is still being processed (by design, so you always see which source is active).

A success message appears when processing completes. If no data is found for a source, it is silently skipped.

7.2 Inspect Data (teal button)

The teal **"Inspect data"** button stops before signal extraction and plot building. It is the **recommended first step** when opening a new patient folder: no figures are built, so it is faster, and it immediately exposes loading errors, missing folders, or unexpected column names — before you run the heavier visualization. The same progress bar is shown, colored teal.

When inspection completes, a **full-screen inspection modal** opens, with one section per datasource containing:

- **Status badge** — colored per outcome:
  - green `ok` — data loaded successfully
  - orange `file_not_found` — no matching file/folder in the patient directory
  - red `load_error` or `format_error` — loading or formatting raised an exception (the error message is shown below the badge)
- **File path** — the detected data file or folder.
- **Date ranges** — two lines. *Date range in file* is the source's own timestamps, untouched. *After time options* is what the application will actually plot, once **every** time setting has been applied: the source's time shift, its recording start or day for sources that need one, its timezone, and finally the `datetime_start` / `datetime_end` window. The second line differing from the first is therefore normal and not necessarily a sign that data was cut.
- **Columns table** — each column with: raw name, whether it is configured in the database options, the point count in the file, the count kept after the same time options, and the first and last kept timestamps.

  Note for the Other (Generic) source: because the `other` folder may contain several independent files, the inspection modal shows **one entry per file** (e.g. `other::waves`, `other::numerics`) rather than a single aggregated entry. Each entry has its own date range and column list.

A **“Download CSV”** button in the modal header exports the full inspection result as a CSV for offline analysis, sharing, or import into the `generate-database-options` helper.

7.3 Inspecting only the signals you configured

Once your database options are settled, the unconfigured columns are usually noise, and reading them costs memory you may not have on a long recording. The setting **“Inspect: read only configured signals”** (Settings → App behavior) makes Inspect read only the signals listed in your database options. A section built that way says so above its table, so a column you do not see is one you have not configured — never one missing from the data.

A small **“Configured columns only: on/off”** label sits next to the Inspect data button itself, mirroring the setting so it stays visible even with the Settings modal closed — a safeguard against running a search for a signal you expect to see, on a setting you forgot was on.

Two things stay true whatever you set here:

- **The time window is never applied while reading.** Inspect’s whole point is to compare the file against the window you asked for — how many points survive it, and whether the file even covers it. Reading the window directly would make every source look like a perfect 100% match, and the single most common problem Inspect catches (a window off by an hour, about to plot forty points) would become invisible.
- **The saving only ever applies to a parquet read.** For most sources that means a patient you have already processed once: the first run has to read the manufacturer’s export as it comes, and only the copy the application then saves in `clinical_scope_output/` can be read selectively. On a first-ever inspection of those sources the setting changes nothing, and the table shows every column as usual. The **Other (Generic)** source is the exception — it reads its own CSV/parquet files directly rather than through that cache, so a `.parquet` file there is pruned starting on the very first inspection (a `.csv` file there still shows every column, whatever this setting is).

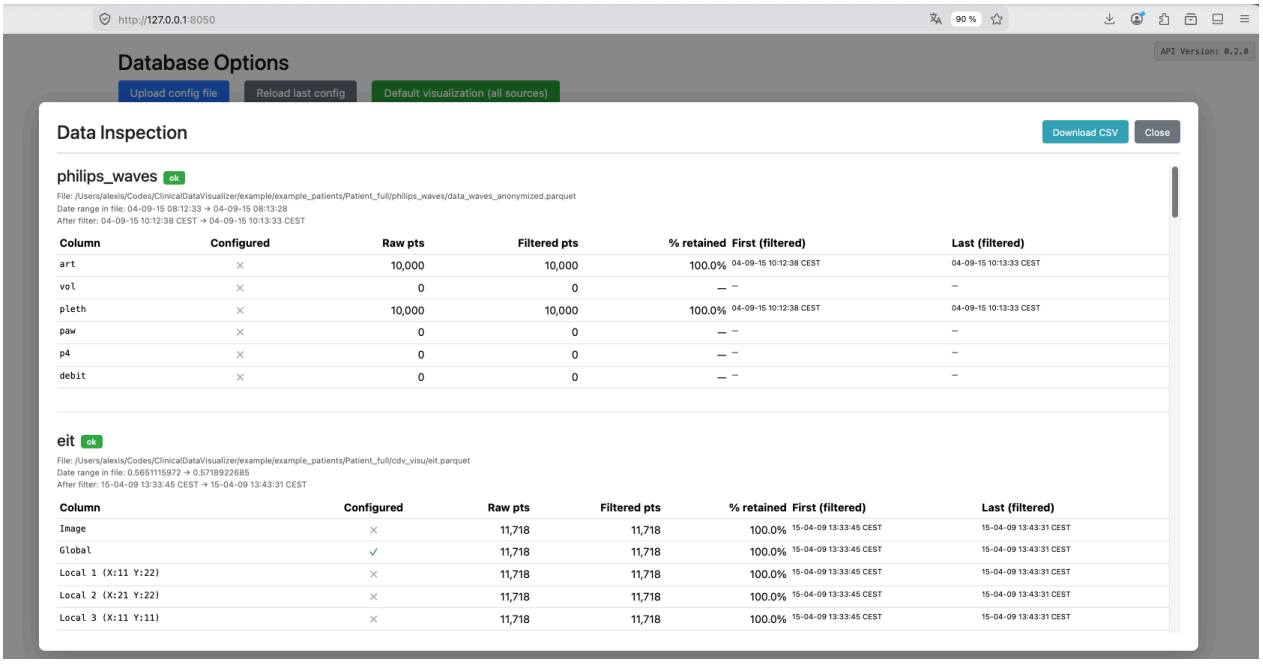


Figure 7: Inspect data pop up

7.4 Inspecting from the Command Line

The same inspection is available as a standalone script:

```
python scripts/inspect_patient_data.py <patient_folder> \
    [--database-options <path>] \
    [--patient-options <path>] \
    [--output-csv out.csv] \
    [--configured-columns-only] \
    [--verbose]
```

Without `--database-options`, all registered datasources are inspected with their defaults. Pass `--output-csv` to save the same per-column table the UI download produces. `--configured-columns-only` is the command-line form of the Settings option above, with the same two caveats.

8 Interacting with Plots

8.1 Navigation Controls

Each plot provides a toolbar (top-right corner) with the following tools:

Tool	Action
Zoom	Click and drag to zoom into a rectangular region
Pan	Click and drag to move the view
Zoom In / Zoom Out	Incremental zoom buttons
Autoscale	Reset the view to fit all data
Reset Axes	Return to the original view
Download as PNG	Save the current plot view as an image

Two shortcuts need no toolbar: the **scroll wheel** zooms the x-axis, and a **double-click** resets the axes to show all data.

8.2 Dynamic Resampling (FigureResampler)

For high-frequency signals (e.g., waveforms sampled at hundreds of Hz), the application uses **Plotly-Resampler** to dynamically load detail as you zoom in. A long time range shows a downsampled overview; zooming into a shorter window loads the full-resolution data automatically. This keeps the interface responsive with millions of data points.

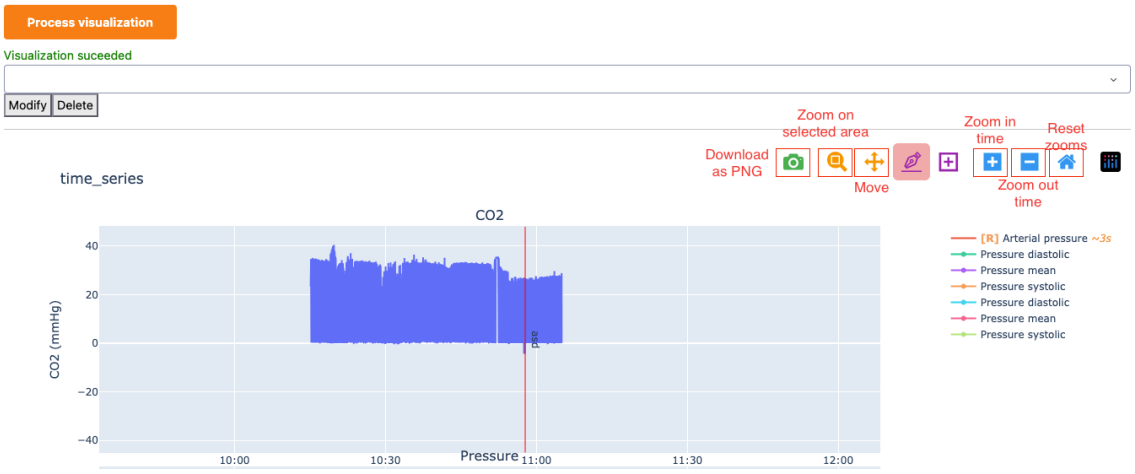


Figure 8: Interactive plot navigation

9 Annotations

The annotation system lets you mark events, time windows, or individual points directly on the plots. Annotations are saved to `annotations.json` in the patient folder and persist across sessions.

9.1 Annotation Toolbar

After a successful visualization, the **annotation toolbar** appears above the plots. It contains:

- **Type buttons** — select the annotation type to place next: *Time Event*, *Time Window*, or *Point*.
- **New Group** — create a named group to organise related annotations.
- **Drag** — switch to nudging existing annotations with the mouse instead of placing new ones.
- **Active group display** — shows which group new annotations will belong to.
- **Save** — writes all annotations to disk.
- **Exit mode** — visible whenever a mode is on; click to leave it without saving.

9.2 Annotation Types

Type	Description	Supported plots
Time Event	Vertical line marking a single instant	Time-axis plots only
Time Window	Shaded region spanning a time range	Time-axis plots only
Point	Dot marker at a specific (x, y) location	All plots including loop plots

Click a type button to activate it, then click (or click-and-drag for Time Window) on a plot. A creation modal appears where you can set a **label** and **color** before confirming.

Spectrograms have a time axis like a time-series plot, so all three types can be placed on them. Loop and PSD plots take Point annotations only, because their x-axis is a signal value and a frequency respectively, not a time.

9.3 Moving an Annotation

Each annotation in the list below the toolbar carries a **Move to...** button. Click it, then click the new position on the plot — a Time Window takes two clicks, the new start and then the new end, exactly like placing one. The annotation keeps its label, colour and group; only its position changes. An annotation placed on all subplots stays on all subplots.

Starting a move leaves the active group, so annotations you place afterwards are ungrouped until you click **Continue** on that group again. **Exit mode** cancels a move you have started but not finished.

Use **Move to...** whenever the annotation has to end up somewhere new — on a different subplot, or on a different plot. To correct a mark that is only slightly off, **Drag** below is quicker.

9.4 Dragging an Annotation with the Mouse

Drag turns the annotations already on the plots into handles you can grab: take a Time Window's edge to stretch it, its body to slide it whole while keeping its length, a Time Event to move it, or a Point to reposition it. For a mark that is a few seconds out, this is quicker than **Move to...**.

It is a mode you switch on and off, because while it is on the annotations sit over the plots and take the clicks and hover that would otherwise reach the signals underneath — click **Drag** again, or **Exit mode**, when you are done. Placing new annotations and dragging existing ones cannot both be on: turning one on turns the other off.

A drag only ever changes position along the time axis, and a Point's value. Dragging up or down does nothing — release the mouse and the annotation snaps back to where it belongs. Use **Move to...** to put an annotation on a different subplot. One thing to watch: a Point on a loop plot carries the time it was taken from, and a drag cannot work that time out again, so a dragged Point loses it. The annotation list shows each Point's time, so you can see when that has happened.

9.5 Hiding an Annotation

A busy plot — a group of several hundred time windows, say — can be harder to read than the signals underneath it. Two buttons on each annotation in the list control how much of it is drawn:

- **L:on / L:off** hides just the text label, leaving the line, rectangle or point marker in place. Useful when you want to see *where* your marks are without reading them.
- **V:on / V:off** hides the annotation completely — label, shape and marker alike. Hiding a large group also makes the plot redraw noticeably faster.

Group headers carry the same pair, **Labels: on/off** and **Visible: on/off**, which apply to every annotation in the group at once. If only some members are hidden, the group reads as shown, so one click hides all of it.

While an annotation is hidden its label and **Move to...** buttons are greyed out — their effect would not be visible until you show it again. Hiding never deletes anything: the annotation stays in the list, the count above it reports how many are hidden, and showing it again restores exactly the label setting it had.

Visibility is saved with everything else, so a hidden group stays hidden the next time you open the patient. If you open a file and the plot looks empty, check the annotation count — it will tell you how many are hidden.

9.6 Groups

Annotations can be organised into named groups. Click **New Group**, enter a name, and all subsequent annotations will belong to that group until you switch or create another. A group is not saved as an entity of its own — each annotation carries its group's name, so an empty group does not survive a save.

9.7 Persistence

`annotations.json` sits in the patient data folder, next to the datasource sub-folders. Annotations are reloaded automatically when you re-process the same patient.

If you write the file by hand or generate it from another tool, any extra fields you add to an annotation are kept as they are.

9.8 Python API

Annotations can be loaded programmatically for analysis:

```
from clinical_scope import load_annotations, load_database_annotations

# Single patient - accepts a JSON file, a clinical_scope_output/ folder, or a patient folder
annotations = load_annotations("/data/Patient01")

# Whole database - scans all patient sub-folders and sets ann.patient on each result
all_anns = load_database_annotations("/data")
```

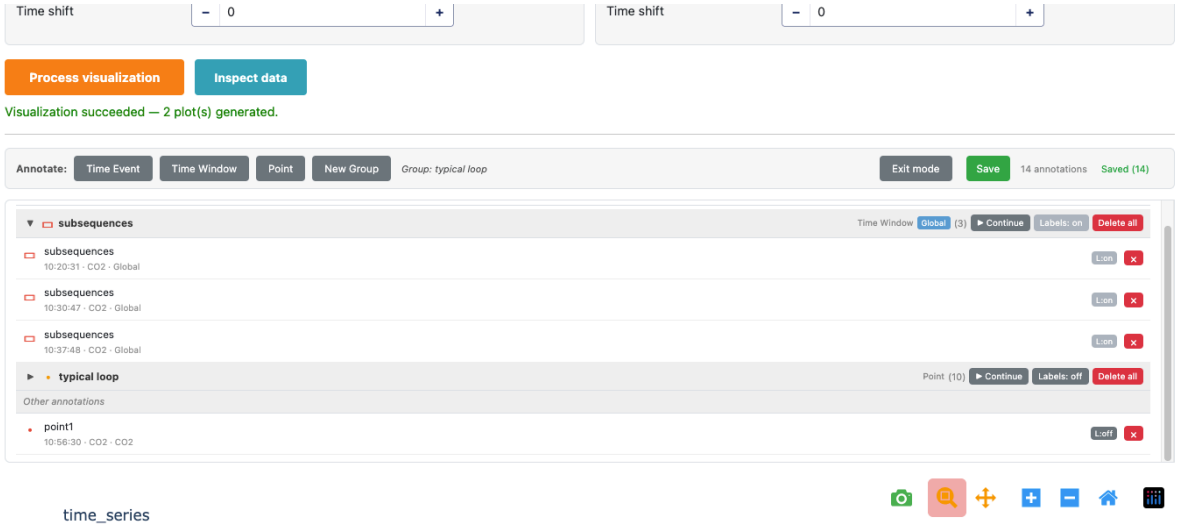


Figure 9: Annotations tools

10 Configuration File Reference

10.1 patient_options.json

This file defines patient-specific settings. It is automatically saved to the `clinical_scope_output/` subfolder each time you click "Process visualization".

```
{
    "data_folder": "/path/to/patient/data",
    "datetime_start": "2024-10-08 10:00:00",
    "datetime_end": "2024-10-08 12:00:00",
    "quick_load": false,
    "other::numerics": {
        "time_shift": 20.0
    },
    "eit": {
        "day": "2024-10-08"
    },
    "edf": {
        "recording_start": "2024-10-08 08:12:33"
    },
    "other::waves": {
        "time_shift": 5.0,
        "group_by_file": false
    }
}
```

Key	Type	Default	Description
`data_folder`	string	—	Path to the patient's root data folder (required)
`output_root`	string	`""`	Writable folder for output when the data folder is read-only. Leave empty to write inside the patient folder. When set, all output goes to `<output_root>/<patient_folder_name>`
`datetime_start`	string or null	null	Start of the time window (`YYYY-MM-DD HH:MM:SS`). Leave empty to use all available data.
`datetime_end`	string or null	null	End of the time window. Leave empty to use all available data.
`quick_load`	boolean	false	Reuse previously cached `.parquet` files in `clinical_scope_output/`
`<source_name>`	object	—	Per-source options block (e.g., `time_shift`, `day`)

Key	Type	Default	Description
`other::<stem>`	object	—	Options for a single file inside `other/`, taking precedence over the shared `other` block. See Generic "Other" Data Source.

`output_root` (read-only data folders). Set this when the patient folder lives on a read-only mount and ClinicalScope cannot write its cache, annotations, or saved configs in place. The layout you already know is reproduced one level deeper, so the root ends up holding one subfolder per patient and `load_database_annotations("<output_root>")` and batch `save_folder` reads work unchanged. Use **one `output_root` per set of patients** — two patient folders sharing a name (e.g. `patient_01`) under the same root would collide.

10.2 database_options.json

This file controls which data sources are active and how each signal is displayed. A snapshot is automatically saved to `clinical_scope_output/database_options.json` each time you click "Process visualization".

10.2.1 Top-Level Structure

```
{
    "global": {
        "grouped_fields": { "Pressure": ["other::waves::ART", "servo_u::Airway Press. (cmH2O)"] }
    },
    "other::waves": { ... },
    "servo_u": { ... },
    "eit": { ... }
}
```

Each data source key is optional — only include the sources you want to enable. The presence of a source key in this file is what activates that source; removing it disables it entirely.

10.2.2 Per-Source Block Structure

```
"other::waves": {
    "field_display": ["ART", "PAP", "P-aer"],
    "signals": {
        "ART": {
            "label": "Arterial pressure",
            "unit": "mmHg",
            "unit_conversion": 1.0,
            "range": [-10, 200],
            "priority": 1.0,
            "color": "red",
            "visible": true,
            "line_dash": "solid",
            "period_resampling": 0.2,
            "hover_template": null
        }
    },
    "grouped_fields": {
```

```
        "Respiratory": ["P-aer", "CrbVol"]
    },
    "loop": {
        "pv_loop": ["P-aer", "CrbVol"]
    },
    "spectrogram": {
        "Fp1 spectrogram": {
            "signal": "Fp1",
            "freq_range": [0.5, 30.0],
            "db_range": [0, 40]
        }
    },
    "numerics": {
        "period_resampling": 0.5,
        "priority": 1.0
    },
    "trace_options": {
        "mode": "lines+markers",
        "line_width": 2.0
    },
    "additional_informations": {
        "timezone": "Europe/Paris"
    }
}
```

10.2.3 Per-Source Fields Reference

Key	Type	Default	Description
`field_display`	list of strings	all signals	Signal names to display. Signals absent from this list are loaded but hidden. Omit to show all.
`signals`	object	`{}`	Per-signal display options (see Per-Signal Fields Reference below).
`grouped_fields`	object	`{}`	Groups of signals to overlay on the same subplot, within this datasource. `{"Respiratory waves": ["signal_1", "signal_2", ...], ...}`
`loop`	object	`{}`	PV-loop definitions: `{"loop_name": ["x_signal", "y_signal"], ...}`.
`spectrogram`	object	`{}`	Spectrogram definitions (see `spectrogram` below).

Key	Type	Default	Description
`psd`	object	`{}`	Power spectral density definitions (see `psd` below).
`numerics`	object	`{}`	Datasource-level defaults applied to every signal (see `numerics` below).
`trace_options`	object	source default	How every trace of this datasource is drawn — line, markers, opacity (see `trace_options` below).
`additional_informations`	object	`{}`	Device-level metadata, including timezone override (see `additional_informations` below).

10.2.4 Per-Signal Fields Reference (`signals.<signal_name>`)

Key	Type	Default	Description
`label`	string	signal name	Display label shown on plot axes and legends.
`unit`	string	`""`	Unit string shown on the Y-axis (e.g., `"mmHg"`, `"cmH2O"`).
`unit_conversion`	float	`1.0`	Multiplication factor applied to raw values for unit conversion. On a derived signal, it multiplies the derived signal's own result instead, and a value equal to its `source`'s is ignored as copied.
`range`	`[min, max]` or null	auto	Fixed Y-axis range. Either bound can be `null` for auto-scaling.

Key	Type	Default	Description
`priority`	float	source default	Plot ordering priority (lower = higher on page). It only orders subplots and never puts two signals on the same one — use `grouped_fields` (the `groups` column in Excel) for that. A group is placed at the average priority of its members, and a member with no priority, here or in `numerics`, counts as a very large value: one such member pulls its group towards the bottom of the page. To place a group predictably, give all its members the same priority.
`color`	string	auto	Line color (any CSS color string, e.g., `"red"`, `"#1f77b4"`).
`visible`	boolean	`true`	Set to `false` to load the signal but hide it from the plot by default.
`line_dash`	string	`"solid"`	Line style: `"solid"`, `"dash"`, `"dot"`, `"dashdot"`.
`period_resampling`	float	source default	Resampling period in seconds for this specific signal. Ignored on a derived signal.
`hover_template`	string	`null`	Custom hover tooltip. Magic keywords: `"fraction"` shows values in (0, 1) as `1/n`; `"percentage"` shows them as `33.3%`. Any other string is passed directly to Plotly as a `hovertemplate`. Leave empty for the default compact display.
`source`	string	`null`	Makes this entry a derived signal: the raw name of the signal, from the same data source, that it is computed from — a signal of the data, or another derived signal.

Key	Type	Default	Description
`transform`	object	`null`	How a derived signal is computed from its `source`: the method, then its parameters, e.g. `{"method": "lowpass", "cutoff_hz": 5}`.

10.2.5 `numerics` Block (Datasource-Level Defaults)

The `numerics` block sets **default values** for every signal of a datasource without listing each one — despite the name, it does not select only the numeric ones. Any per-signal entry inside `signals` takes precedence.

```
"other::numerics": {
    "numerics": {
        "period_resampling": 0.5,
        "priority": 2.0
    }
}
```

Key	Type	Default	Description
`period_resampling`	float	source default	Resampling period in seconds, applied to every signal in this datasource.
`priority`	float	source default	Default plot priority for every signal of this datasource (lower = higher on page). Overridden per signal by `signals.<name>.priority`.

> In the Excel format, these values are set via the **sentinel row** (`signal = *`). See database_options.xlsx.

10.2.6 `trace_options` Block (Datasource-Level Trace Style)

The `trace_options` block changes how every trace of a datasource is drawn. It works in any per-source block — a device datasource, or an `other::<stem>` file scope — and each key you set replaces the datasource's built-in default while the keys you leave out keep theirs.

A dense overlay reads better semi-transparent — the demo database draws the EIT impedance curves this way, so the individual regions stay legible where they cross:

```
"eit": {
    "trace_options": { "opacity": 0.7 }
}
```

Key	Type	Default	Description
`mode`	string	`"lines"`	`"lines"`, `"markers"` or `"lines+markers"`.
`line_width`	float	source default	Line width in pixels.

Key	Type	Default	Description
`line_dash`	string	`"solid"`	Line style: `"solid"`, `"dash"`, `"dot"`, `"dashdot"`.
`opacity`	float	`1.0`	Trace opacity, 0–1.
`marker_symbol`	string	source default	Plotly marker symbol (e.g. `"circle"`, `"square"`), used when `mode` includes markers.
`marker_size`	float	source default	Marker size in pixels.

A key you misspell is reported when the configuration is checked, and ignored. Per-signal `color`, `line_dash` and `visible` live in the `signals` block and win over anything set here — use `trace_options` for the whole datasource and `signals` for the exceptions.

> In the Excel format, these values are set via the **sentinel row** (`signal = *`), in the `trace_mode`, `line_width`, `opacity` and `marker_symbol` columns. See database_options.xlsx.

10.2.7 `spectrogram` Block

The `spectrogram` block defines time-vs-frequency-vs-power plots — one entry produces one subplot, a heatmap with time on the x-axis, frequency on the y-axis, and power as colour. EEG is the main use case, but any sufficiently sampled signal works.

```
"edf": {
    "spectrogram": {
        "Fp1 spectrogram": {
            "signal": "Fp1",
            "freq_range": [0.5, 30.0],
            "db_range": [0, 40]
        }
    }
}
```

Key	Type	Default	Description
`signal`	string	— (required)	Raw name of the one signal to analyze. No arithmetic — pair or aggregate signals in the source data first if needed.
`freq_range`	`[min_hz, max_hz]`	— (required)	Frequency band to display. There is no workable default — pick the band relevant to the signal (e.g. 0.5–30 Hz for EEG).

Key	Type	Default	Description
`db_range`	`[min_db, max_db]` or null	your Settings	Fixed colour scale bounds. Left unset, it falls back to your personal colour range setting — fixed rather than auto-scaled, so appearance stays comparable across patients.
`window_s`	float	derived from `freq_range`	Advanced: override the analysis window length in seconds. Leave unset unless you know you need it.
`overlap`	float	`0.5`	Advanced: override the fraction of overlap between consecutive analysis windows. Leave unset unless you know you need it.

A signal whose `period_resampling` decimates it (see Per-Signal Fields Reference) cannot be turned into a spectrogram — decimation has no anti-aliasing filter, so the result would show rhythms that are not really there. Remove `period_resampling` for that signal to enable its spectrogram; the log explains which signal and why when this happens.

Power is shown as a spectral density in decibels, so the level a signal reads at does not change with `window_s` or with how fast the signal was sampled. One `db_range` therefore stays meaningful across channels recorded at different rates, and two window lengths of the same channel can be compared directly. Window shape, detrending and averaging follow the defaults of the standard Welch method, so the same numbers come out of any reference implementation.

10.2.8 `psd` Block

The `psd` block defines power-vs-frequency plots — frequency on the x-axis, power in dB on the y-axis, averaged over the whole loaded time range. Where a spectrogram shows how a rhythm evolves, a PSD shows the shape of the spectrum at rest. One entry produces one subplot, with **one line per signal listed** so several channels can be compared side by side. Typical uses: frequency-domain heart-rate variability, separating ventilation from cardiac components in EIT, and EEG band content.

```
"edf": {
    "psd": {
        "EEG PSD": {
            "signals": ["chan 1", "chan 2", "chan 3"],
            "freq_range": [0.5, 30.0],
            "db_range": [0, 40]
        }
    }
}
```

Key	Type	Default	Description
`signals`	list	— (required)	Lines to overlay on this plot — see below for the two ways to write one.
`freq_range`	`[min_hz, max_hz]`	— (required)	Frequency band to display. There is no workable default — pick the band relevant to the signal (e.g. 0.5–30 Hz for EEG, 0.04–0.4 Hz for heart-rate variability).
`db_range`	`[min_db, max_db]` or null	auto	Fixed bounds for the power axis. Left unset, the axis scales to the data.

To compute a PSD over part of a recording rather than all of it, narrow the run with the **Time start** / **Time end** filters — a PSD always covers exactly the time range that was loaded.

Each line takes the colour of its signal, so it matches that signal's time-series trace. The `period_resampling` restriction described for spectrograms above applies here too, and refusing one signal refuses the whole plot: a comparison missing one of its channels invites a wrong reading more than an absent plot does. The power axis is the same spectral density described for spectrograms above, which is what lets two lines with different `window_s` values sit on one plot.

Writing a `signals` entry. Most of the time a plain name is enough — it is resolved exactly like `grouped_fields` (see Signal Reference Resolution). Write an object instead when you want to compare the *same* signal analyzed two different ways on the same plot — e.g. a short vs. a long analysis window, to see whether a longer window is smearing two close frequency peaks together:

```
"psd": {
    "EEG PSD": {
        "signals": [
            {"signal": "chan 1", "window_s": 2.0, "label": "narrow window"},
            {"signal": "chan 1", "window_s": 8.0, "label": "wide window"}
        ],
        "freq_range": [0.5, 30.0]
    }
}
```

Key	Type	Default	Description
`signal`	string	— (required)	The signal for this line, resolved the same way as a plain-string entry.
`window_s`	float	derived from `freq_range`	Advanced: override the analysis window length in seconds for this line only.
`overlap`	float	`0.5`	Advanced: override the fraction of overlap between consecutive analysis windows for this line only.

Key	Type	Default	Description
`label`	string	the signal's own name	What this line is called on hover. **Required** when the same signal appears more than once in one plot — otherwise the two lines are indistinguishable.
`color`	string	the signal's own colour	Line colour for this line only. Two lines from the same signal otherwise inherit the same colour and overlap indistinguishably.
`line_dash`	string	the signal's own dash style	`solid`, `dash`, `dot`, `dashdot` for this line only — a second way (besides colour) to tell two lines from the same signal apart.

10.2.9 `additional_informations` Block

The `additional_informations` block carries device-level metadata that affects how raw data is interpreted. Currently its only field is `timezone`.

```
"eit": {
    "additional_informations": { "timezone": "UTC" }
}
```

Key	Type	Default	Description
`timezone`	string	source default	Override the timezone used to give a timezone (e.g., `"Europe/Paris"`, `"UTC"`) to timestamps which are timezone-naive. All plots still render in the configured display timezone.

All datasources apply timezone according to the same rule:

- If the loaded data already carries timezone information, it is kept as-is.
- If the data is timezone-naive, the timezone is resolved in this order:
  1. `additional_informations.timezone` in the database options (if present)
  2. The datasource's built-in default timezone

  In the Excel format, set the timezone via the **sentinel row** (`signal = *`, `timezone` column). See database_options.xlsx.

10.2.10 Global Fields

```
"global": {
    "grouped_fields": {
        "Pressure": ["ART", "PNIs", "PNIm", "PNId"]
    },
    "loop": {
        "PV loop (ventilator vs. FluxMed)": [
            "servo_u::Paw", "fluxmed_signals::Vt"
        ]
    }
}
```

Key	Description
`global.grouped_fields`	Groups signals from **different** datasources onto the same subplot. Signal names must be unique across the datasources involved — use `datasource::raw_name` to disambiguate.
`global.loop`	Cross-datasource phase loops: `{"loop_name": ["x_ref", "y_ref"]}`. Both signals can come from different datasources. Each `*_ref` is resolved by the same three-mode chain as `grouped_fields`.

10.2.11 Signal Reference Resolution

Signal references in `grouped_fields`, `loop` and `psd` are resolved by the following three-mode lookup chain — in `global.grouped_fields` and `global.loop`, and in the per-source blocks alike:

1. **Qualified reference** `datasource::raw_name` — explicit and unambiguous. Recommended when the same column name exists in several datasources.
2. **Display name** — the `label` from the signals block. Works when the label is unique across sources; a warning is logged if it matches several signals.
3. **Raw name fallback** — the column name as it appears in the raw data file.

A signal from a file inside `other/` can be written either way: `other::waves::art` is the fully qualified form, and `waves::art` is the raw-name form — both reach the same signal.

One consequence: if a file inside `other/` is named after a data source — `other/servo_u.parquet` — then `servo_u::Paw` could mean either that file's `Paw` column or the real servo-u one. The data source always wins, and the log tells you so and gives you the fully qualified spelling (`other::servo_u::Paw`) that reaches the file's column instead.

Multi-cycle loops are rendered with a **time-range slider** below the plot so you can scroll through cycles.

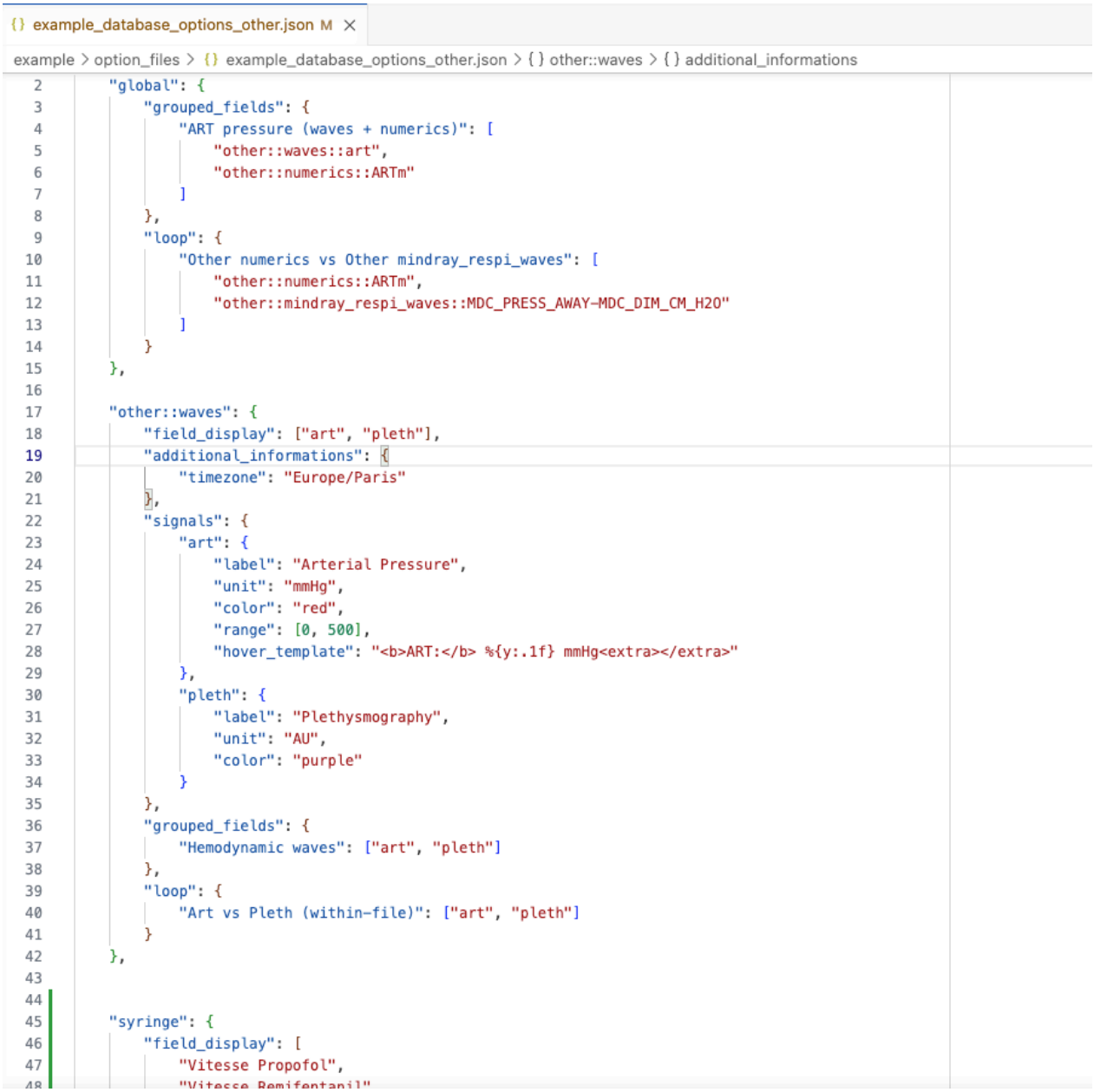


Figure 10: example json database options file

10.3 database_options.xlsx

For clinical users, the **Excel format is the recommended way** to configure signals — it requires no knowledge of JSON and can be edited in any spreadsheet application. On upload, it is automatically converted to the equivalent JSON structure, so every option available in JSON is also available in the spreadsheet.

The file must contain a sheet named **signals**, and may add **loops**, **spectrograms** and **psds**. An optional sheet that is absent or malformed is silently skipped.

10.3.1 signals sheet

One row per signal. The columns datasource and signal are mandatory; all others are optional and fall back to the defaults listed in the per-signal table above.

Use * in the signal column to write a **sentinel row** that sets datasource-level defaults (e.g., a common period_resampling or timezone) without defining a specific signal — equivalent to the numerics and additional_informations blocks in JSON. Column names are case-insensitive (e.g., Label, UNIT, Hover_Template all work).

The **Scope** column below indicates where each field is meaningful:

- **Both** — valid in sentinel (*) and per-signal rows
- **Signal** — per-signal rows only; ignored in sentinel rows
- **Sentinel** — sentinel (*) rows only; a warning is logged if set in a per-signal row

Column	Required	Scope	Description
datasource	Yes	Both	Data source name (e.g., servo_u, eit, other::waves).
signal	Yes	Both	Raw signal name. Use * for a sentinel row that sets datasource-level defaults.
label	No	Signal	Display label. Defaults to the signal name if empty or identical.
unit	No	Signal	Unit string (e.g., mmHg).
unit_conversion	No	Signal	Numeric multiplier for unit conversion. On a derived signal, it multiplies the derived signal's own result instead, and a value equal to its source's is ignored as copied.
range_min	No	Signal	Minimum Y-axis value.
range_max	No	Signal	Maximum Y-axis value.

Column	Required	Scope	Description
`priority`	No	Both	Plot priority (float; lower = higher on page). In a sentinel row sets the datasource-level default; in a signal row overrides it for that signal only. It orders subplots and never merges them — use `groups` for that (see priority).
`color`	No	Signal	CSS color string.
`visible`	No	Signal	`yes` / `no` (default: `yes`). Set `no` to draw the signal but start it hidden — it stays in the legend, and one click brings it back. Accepts `yes`, `1`, `true`, `oui`, `vrai` (case-insensitive).
`line_dash`	No	Signal	`solid`, `dash`, `dot`, `dashdot`.
`period_resampling`	No	Both	Resampling period in seconds. In a sentinel row sets the datasource-level default; in a signal row overrides it for that signal only.
`hover_template`	No	Signal	Hover tooltip format. Magic keywords: `"fraction"` shows values in (0, 1) as `1/n`; `"percentage"` shows them as `33.3%`. Any other string is forwarded directly to Plotly as a `hovertemplate`.

Column	Required	Scope	Description
`display`	No	Signal	`yes` / `no` — whether to add this signal to the display list. Default: `yes`. Set `no` to keep the row's label and unit on file while leaving the signal out of the plots entirely: unlike `visible`, it produces no trace and no legend entry. Useful for parking a signal you may want back later, or for describing a column that is not worth plotting (see `Comments(-)` under `fluxmed_parameters` in `example/demo_database/database_opt`
`groups`	No	Signal	Semicolon-separated group names (e.g., `Respiratory;Pressure`). Groups within one datasource become local `grouped_fields`; groups spanning multiple datasources become `global.grouped_fields`.
`source`	No	Signal	Raw name of the signal this row is computed from, in the same `datasource` — a signal of the data, or another derived row. Makes the row a derived signal.
`transform`	No	Signal	How to compute it: the method, then its parameters by name, e.g. `lowpass(cutoff_hz=5, order=4)`. See Derived signals.
`timezone`	No	**Sentinel**	Override the timezone for this datasource (e.g., `"Europe/Paris"`, `"UTC"`). Only valid in `*` rows; a warning is logged if placed in a per-signal row. Works with `other::<stem>` datasource keys. See `additional_informations` Block for which datasources support this.

Column	Required	Scope	Description
`trace_mode`	No	**Sentinel**	`lines`, `markers` or `lines+markers` for every trace in this datasource, `other::<stem>` keys included — e.g. a sparse infusion log reads better as `markers` than as connected `lines`. Only valid in `*` rows; see `trace_options`.
`line_width`	No	**Sentinel**	Line width in pixels for every trace in this datasource. Only valid in `*` rows.
`opacity`	No	**Sentinel**	Trace opacity, `0-1`. Only valid in `*` rows.
`marker_symbol`	No	**Sentinel**	Plotly marker symbol (e.g., `circle`, `square`) used when `trace_mode` includes `markers`. Only valid in `*` rows.

10.3.2 `loops` sheet (optional)

One row per PV-loop definition — equivalent to the `loop` key in the JSON per-source block or in `global`.

Column	Required	Description
`datasource`	Yes	Data source that owns both signals.
`loop_name`	Yes	Name for the loop plot (e.g., `pv_loop`).
`x_signal`	Yes	Signal name for the X axis.
`y_signal`	Yes	Signal name for the Y axis.

10.3.3 `spectrograms` sheet (optional)

One row per spectrogram definition — equivalent to the `spectrogram` key in the JSON per-source block.

Column	Required	Description
`datasource`	Yes	Data source that owns the signal.
`spectrogram_name`	Yes	Name for the spectrogram plot (e.g., `Fp1 spectrogram`).
`signal`	Yes	Raw name of the signal to analyze.
`freq_min`	Yes	Minimum frequency to display (Hz).
`freq_max`	Yes	Maximum frequency to display (Hz).
`db_min`	No	Colour scale minimum (dB). Leave both `db_min` and `db_max` empty to use your personal colour range setting.

Column	Required	Description
`db_max`	No	Colour scale maximum (dB). Must be set together with `db_min`, or both are ignored.
`window_s`	No	Advanced: override the analysis window length in seconds. Leave unset unless you know you need it.
`overlap`	No	Advanced: override the fraction of overlap between consecutive analysis windows. Leave unset unless you know you need it.

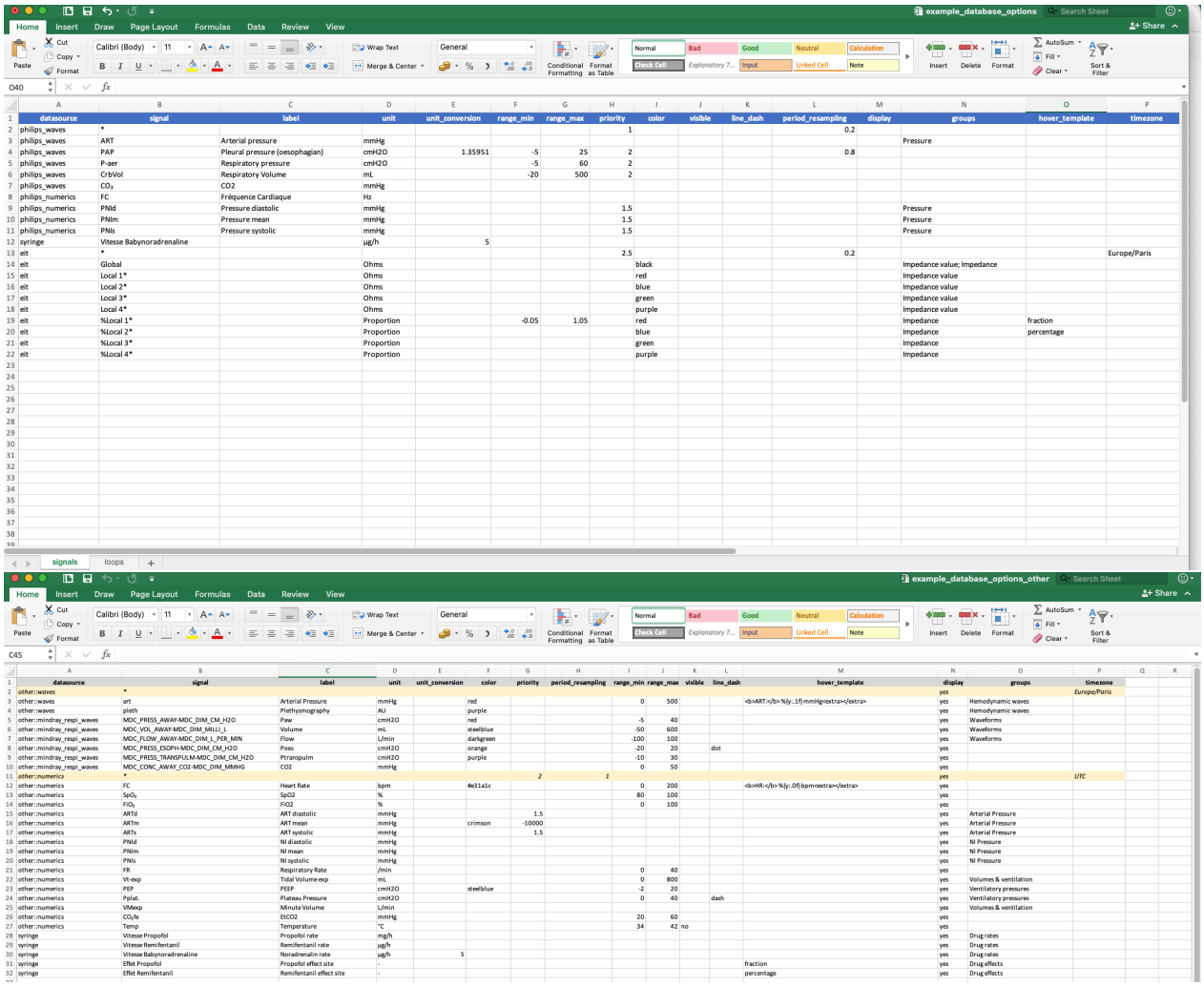
10.3.4 `psds` sheet (optional)

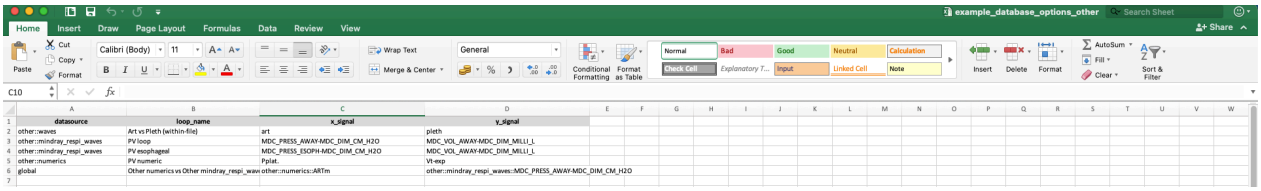
One row per signal — equivalent to the `psd` key in the JSON per-source block. A signal can belong to zero, one, or several PSD plots, just like the `groups` column on the `signals` sheet. Signals sharing a group are overlaid on the same plot, one line each.

Column	Required	Description
`datasource`	Yes	Data source that owns the signal.
`groups`	Yes	Semicolon-separated PSD plot names (e.g., `EEG PSD;Low band`). Each name becomes its own plot; signals sharing a name overlay on it. Leave empty to exclude the signal from every PSD plot.
`signal`	Yes	Raw name of the signal for this line.
`freq_min`	Yes	Minimum frequency to display (Hz). Read from the first row that introduces each plot; a later row with a different value logs a warning and is ignored.
`freq_max`	Yes	Maximum frequency to display (Hz). Same first-row-wins rule as `freq_min`.
`db_min`	No	Power axis minimum (dB). Leave both `db_min` and `db_max` empty to scale the axis to the data. Same first-row-wins rule as `freq_min`.
`db_max`	No	Power axis maximum (dB). Must be set together with `db_min`, or both are ignored.
`window_s`	No	Advanced: override the analysis window length in seconds for this row's line only. Leave unset unless you know you need it.
`overlap`	No	Advanced: override the fraction of overlap between analysis windows for this row's line only.

Column	Required	Description
`label`	No	What this line is called on hover. **Required** when the same signal appears more than once in one plot (e.g. comparing two `window_s` values) — otherwise the two lines are indistinguishable.
`color`	No	Line colour for this row's line only. Leave unset to inherit the signal's own colour — two rows for the same signal then need this to tell their lines apart.
`line_dash`	No	`solid`, `dash`, `dot`, `dashdot` for this row's line only. A second way (besides colour) to tell two lines from the same signal apart.

See `example/demo_database/` in the source repository for a complete example in both formats — `database_options.xlsx` (all four sheets) and `database_options.json`, its exact equivalent. Both configure the shipped `demo_patient/`, so you can load either one and press Process straight away.





datasource	loop_name	x_signal	y_signal
other::waves	Art vs Pleth (within-file)	art	pleth
other::mindray_respi_waves	PV loop	MDC_PRESS_AWAY-MDC_DIM_CM_H2O	MDC_VOL_AWAY-MDC_DIM_MILLI_L
other::mindray_respi_waves	PV esophageal	MDC_PRESS_ESOPH-MDC_DIM_CM_H2O	MDC_VOL_AWAY-MDC_DIM_MILLI_L
other::numerics	PV numeric	Pplat.	Vt-exp
global	Other numerics vs Other mindray_respi_wav	other::numerics::ARTm	other::mindray_respi_waves::MDC_PRESS_AWAY-MDC_DIM_CM_H2O

10.4 Derived signals

A derived signal is a new curve computed from one signal of your data — a smoothed arterial pressure, a filtered EEG channel, a one-minute average of heart rate, the rate of change of a pressure, a signal rescaled to 0–1. It is configured and drawn like any other signal: it has its own label, unit and colour, it can be grouped with other signals, and it can be used in a loop, a spectrogram or a PSD. Several derived signals can be computed from the same signal, for instance to compare two ways of smoothing it, and a derived signal can itself be computed from another — see Chaining derived signals.

Derived signals appear in the plots only. They are not part of extracted data, and Inspect does not list them.

10.4.1 Adding one

In the `signals` sheet, duplicate the row of the signal you want to compute from, then in the copy:

1. give it a new name in the `signal` column (e.g. `art_lp`) — you refer to it by this name elsewhere, like any raw name;
2. write the original signal's name in the `source` column;
3. write the computation in the `transform` column, e.g. `lowpass(cutoff_hz=0.5)`;
4. change `label`, and perhaps `color` or `line_dash`, so the two curves can be told apart;
5. if the method changes the unit (a rate of change, a logarithm, a normalisation), clear the copied `unit` so it is worked out for you — see What to expect;
6. clear the copied `unit_conversion`, if the original has one: the derived signal is computed from the original as drawn, already converted. A `unit_conversion` equal to the original's is ignored, with a warning when the file is loaded.

The copy keeps the original's `groups`, so it is drawn on the same plot, over the original. The demo configuration in `example/demo_database/` does exactly this for an arterial pressure waveform and for heart rate, and also shows the rate of change of the arterial pressure and a normalised EIT impedance, each drawn over its source with the `unit` left empty.

In JSON, the same derived signal is an entry of the `signals` block, with `transform` written as an object:

```
"other::waves": {
    "signals": {
        "art": {"label": "Arterial pressure", "unit": "mmHg"},
        "art_lp": {
            "label": "Arterial pressure, low-pass 0.5 Hz",
            "unit": "mmHg",
            "source": "art",
            "transform": {"method": "lowpass", "cutoff_hz": 0.5}
        }
    },
    "field_display": ["art", "art_lp"]
}
```

The `source` must be a signal of the same data source — for `other`, of the same file. It may be another derived signal: see Chaining derived signals.

10.4.2 Methods

Parameters are always written by name, in hertz or seconds, never as a number of samples. The same row therefore gives the same result on a 125 Hz waveform and on numerics recorded once per second. A parameter with a default can be left out.

Method	Parameters	What it does
`lowpass`	`cutoff_hz` (required), `order` (default 4)	Keeps what varies more slowly than `cutoff_hz`: smooths out fast components.
`highpass`	`cutoff_hz` (required), `order` (default 4)	Keeps what varies faster than `cutoff_hz`: removes slow drift.
`bandpass`	`low_hz`, `high_hz` (both required), `order` (default 4)	Keeps only what lies between `low_hz` and `high_hz`.
`bandstop`	`low_hz`, `high_hz` (both required), `order` (default 4)	Removes what lies between `low_hz` and `high_hz`, e.g. 49–51 Hz mains interference.
`moving_average`	`window_s` (required)	Averages over a window of `window_s` seconds centred on each point.
`moving_median`	`window_s` (required)	Takes the median over a window of `window_s` seconds centred on each point: an isolated spike or flush is ignored instead of smeared.
`savgol`	`window_s` (required), `polyorder` (default 2)	Smooths by fitting a polynomial of degree `polyorder` in a window of `window_s` seconds: peaks keep their height better than with a moving average.
`derivative`	`window_s` (default 0)	Rate of change per second. With the default `0` it is the plain difference between neighbouring points; a larger `window_s` smooths it over that many seconds.
`log10`, `ln`	none	Base-10 or natural logarithm. Zero and negative values leave a gap.
`exp`	none	Exponential. A value too large to represent leaves a gap.
`abs`	none	Absolute value.
`negate`	none	Flips the sign.
`zscore`	none	Rescales to mean 0 and standard deviation 1 (population standard deviation).
`minmax`	`low_pct` (default 0), `high_pct` (default 100)	Rescales so that the `low_pct` percentile is 0 and the `high_pct` percentile is 1. Values beyond them are kept, not clipped, so they fall below 0 or above 1.

A method that takes no parameter is written as its bare name, e.g. `log10`.

The four filters are Butterworth filters of the given `order`, applied forwards then backwards so that the filtered curve does not lag behind the original; a higher order separates kept from removed frequencies more sharply. Before filtering, slightly irregular timestamps are placed on a regular time grid, as they are for `savgol` and `derivative`. The moving average and median, the logarithms, `exp`, `abs`, `negate` and the two normalisations keep the points as recorded. A `cutoff_hz` or `high_hz` must stay below half the signal's sampling rate.

A window is a duration, and one too short for a signal's sampling means "no smoothing" rather than an error: `savgol` returns the signal unchanged, `derivative` uses its three-point minimum, and `moving_median` holds one point. One row therefore runs on a fast waveform and on numerics alike. The log says when this happens.

Rate of change. `derivative` is per second: the derivative of a pressure in mmHg is in mmHg/s. For per minute, set the row's `unit_conversion` to `60` and its `unit` to e.g. `mmHg/min`. A `window_s` of a few samples (for example `0.05` on a 125 Hz waveform) takes the noise out of a derivative.

Normalisation. `zscore` and `minmax` take their statistics from the time window you loaded, between the start and end datetimes. The same row therefore draws a different curve when the window changes. `minmax` never clips, so a single large artefact (such as an arterial line flush) can compress everything else toward one end; use `minmax(low_pct=1, high_pct=99)` together with a `range` such as 0 to 1 to frame the bulk of the signal and let the artefact run off the plot. A constant signal cannot be normalised: it is left out and the log says so.

A mistake the file alone shows — an unknown method, a misspelled or missing parameter, one that is out of range (negative, zero where a positive value is needed, a percentile above 100, `low_pct` not below `high_pct`), a `source` without a `transform` — is reported in the status box when the file is loaded, and that derived signal is not drawn. A misspelled parameter is never silently ignored: the filter would otherwise run with its default, and nothing on the plot would show it.

10.4.3 Chaining derived signals

A derived signal can be the `source` of another: write its name in the second row's `source` column. Each step is then computed from the curve of the step before, as that curve is drawn. In the demo configuration, `art_dpdt` is the rate of change of the arterial pressure (`source art`, `derivative(window_s=0.05)`), and `art_d2` its second derivative (`source art_dpdt`, the same `transform`). Other uses: `lowpass` then `minmax`, to keep the slow, ventilation-driven part of an EIT impedance and rescale it so its shape can be overlaid on volume; or `moving_median` then `derivative`, to take the rate of change of a signal with isolated spikes.

- The rows may come in any order in the file, and a chain may have any number of steps.
- Only displayed rows are drawn. A step you do not want to see need not be displayed: set its `display` to `no` (in JSON, leave it out of `field_display`), and it is still computed as part of every row built from it.
- A chain must lead to a signal of the data. Rows whose sources loop back on each other (`a` computed from `b`, and `b` from `a`) are reported as an error when the file is loaded, and are not drawn.
- If a step cannot be computed for a patient — its cutoff is too high for that recording, say — every row computed from it is left out too, and the log names the step that failed.

10.4.4 What to expect

- **The unit follows the source.** Leave `unit` empty and it is worked out from the unit of the source: unchanged for the filters, the moving average and median, `savgol`, `abs` and `negate`, and rewritten for the others, so the derived curve lands on the right axis when drawn over its source.

Method	Source in `mmHg`	Source with no unit
`derivative`	`mmHg/s`	`/s`
`log10`	`log10(mmHg)`	`log10`
`ln`	`ln(mmHg)`	`ln`
`exp`	`exp(mmHg)`	`exp`

Method	Source in `mmHg`	Source with no unit
`zscore`	`z`	`z`
`minmax`	`norm`	`norm`

In a chain, the unit is worked out step by step: the derivative of the derivative of a pressure in mmHg is `mmHg/s/s`. Type `mmHg/s²` in `unit` if you prefer it.

A `unit` you type always wins. The one exception is a row duplicated from its source: for a method that changes the unit, the copied `unit` is ignored and the derived one is used, with a warning when the file is loaded. A row that sets its own `unit_conversion` must also set `unit`, because the unit cannot be guessed from a factor; without one the unit is `-`, and a warning asks for it. A `unit_conversion` of `1` does not count, since it changes nothing, and neither does one copied from the source, which is ignored as the next point explains.

- **`unit_conversion` scales the derived signal's own result.** A derived signal is computed from its source as drawn, already converted by the source's `unit_conversion`, so the cell is usually left empty. A value you set multiplies the result of the method: `60` turns a rate per second into a rate per minute, and `100` turns the 0–1 of `minmax` into a percentage. Every other empty column (`label`, `color`, `range`, `groups`, …) means its usual default, exactly as on any row.
- **A `unit_conversion` equal to the source's is ignored.** Duplicating the source's row copies its `unit_conversion`, and applying it again would convert the curve twice. So a value equal to the source's own `unit_conversion` is taken as copied and ignored, and a warning says so when the file is loaded. Clear the cell to silence it.
- **`period_resampling` does not apply.** A derived signal is always computed from every recorded point of its source; a `period_resampling` on its row is ignored, with a warning when the file is loaded.
- **Gaps are never bridged.** Where the source has a recording gap, the derived signal has one too: each stretch between gaps is computed on its own (the two normalisations aside, which use every point of the loaded window to set their scale). A stretch too short for the filter is left out, and the log says so.
- **Filters may show transients at the edges.** At the start and end of the time window, and on either side of a gap, a filtered curve can briefly swing away from the original before settling; a smoothing window or a derivative is affected by up to half a window there. This comes with filtering any recording that has a beginning and an end, so read the first and last seconds of a filtered stretch with care.
- **One file, many patients.** If a derived signal cannot be computed for one patient — its source is missing from that patient's data, or its cutoff is too high for that recording's sampling rate — only that derived signal is left out, and the log says why. Everything else is drawn as usual.
- **A real column wins.** If the data already contains a column with the derived signal's name, that column is plotted instead, and the log asks you to rename the row. The same goes for a chain passing through that name: the rows computed from it start from the column.

11 Troubleshooting

11.1 Browser Does Not Open Automatically

If the browser does not open after launching the application, manually navigate to:

`http://127.0.0.1:8050`

Ensure no other application is using port 8050 (typically a previous app launch terminal tab not yet closed). If needed, close the terminal window which was opened in the app and restart the application.

11.2 No Data Found

If the visualization is empty or a data source shows no signals:

- Start by inspecting the data (teal button) rather than visualizing it — it reports much more about what the app could gather from the data.
- Try the default visualization database options, to rule out your own options hiding the data.
- Verify that the **data folder path** is correct and accessible.
- Check that subfolders follow the **naming conventions** (see Section 3).
- Ensure the subfolder contains files with one of the **accepted extensions** for that data source (see Section 3). Files with unrecognized extensions are silently ignored.
- For **single-file** sources: if the folder contains multiple unrelated data files (different stems), the source is skipped. Keep only one data file per folder, or provide the same data in multiple formats (e.g., `data.csv` + `data.parquet`) and the preferred format will be selected automatically.

11.3 Slow Loading

Large datasets may take time to load on the first run. To speed up subsequent loads:

- Enable the **"Re-use data if already loaded once"** (quick_load) option. This uses the cached `.parquet` files in `clinical_scope_output/` instead of re-reading raw data files.

11.4 Time Alignment Issues

If signals from different sources appear misaligned in time:

- Use the **Time shift** option in the per-source settings to adjust alignment.
- Verify that the correct **day** or **date** is set for sources that require it (e.g., EIT).

11.5 Application Crashes or Errors

- Check the terminal window for error messages.
- Log files are available in the `logs/` directory.
- Ensure the data files are not corrupted or truncated.
- If you think you are facing a real bug, please report it on the GitHub issues page.

11.6 Known limitations

These may be tackled in the future, depending on what users need — ask for any of them on the issues page.

- No timeshift inside a datasource, e.g. if 2 timeseries from `servo_u` are not aligned, this currently can't be solved in the app. (Files inside `other/` are the exception — each gets its own `time_shift`.)
- `output_root` keys each patient by its folder name only, so two patient folders with the same name under one `output_root` overwrite each other — see patient_options.json.
- EIT recordings spanning more than one calendar day are unsupported: the device's own files carry no date, so the day is inferred from the **day** option (or, if unset, from **Time start filter**) and applied to the whole recording.

- Spectrograms and PSDs expose only `freq_range`, `db_range`, `window_s` and `overlap`. Everything else about the analysis — window shape, detrending, how windows are averaged, how recording gaps and irregular timestamps are handled — is fixed at the standard Welch defaults and cannot be changed from the config.
- A spectrogram or PSD figure does not state the settings it was drawn with, nor whether anything happened to the data on the way (timestamps regularised, gaps skipped). Keep the `database_options` file alongside the figure if you need that record — for instance to describe the analysis in a publication. The same goes for a spectrogram or PSD of a derived signal: nothing on the figure says it shows filtered data.
- An annotation finds its subplot by title, and a derived signal drawn on a subplot of its own gives that subplot its `label` (or its name, when it has none). Renaming it therefore leaves its annotations behind, as renaming a group does — and a derived signal is likely to be renamed while you try out settings. Settle on its name before annotating it.
- When a derived signal is left out of the plots (source missing, cutoff too high for the recording), the reason is written to the log only; the app itself does not show it.